

Furqan Skills — Implementation Plan v1.0

Building the Two Commercial Products

Project: AI-Furqan → Two Sellable AI Skills **Version:** 1.0 **Date:** March 21, 2026 **Based On:** FURQAN-REASONING-AS-A-SKILL-v1.0.md + FURQAN-MEMORY-SKILL-v1.0.md **Status:** Ready for Execution
Prerequisites: Sprints 3-5 complete (497 tests passing)

Current State

What's Built (Engine — Internal)

src/al_furqan/	
├ engine/	✓ Refactored (axioms, models, prompts, pipeline, gates, chains, Z3)
├ kb/	✓ Collections (Quran 6,236 + Hadith 55 + Fiqh 25) + Graph + Retriever
├ api/	✓ FastAPI + Orchestrator + Auth
├ store/	✓ VerdictStore + FeedbackStore
├ providers/	✓ LLM Layer (Anthropic, DashScope/Qwen, Ollama)
└ 497 tests	✓ All passing

What's Missing (Skills — Commercial)

furqan-raas/	✗ Reasoning-as-a-Skill package
├ mcp_server.py	✗ MCP Server (Model Context Protocol)
├ skill.md	✗ Agent Skill definition
├ adapters/	✗ Knowledge Adapter system
├ api_gateway.py	✗ REST API for remote access
└ sdk/	✗ Python + TypeScript SDKs
furqan-memory/	✗ Memory Skill package
├ mcp_server.py	✗ MCP Server (client-side)
├ skill.md	✗ Agent Skill definition
├ memory_manager.py	✗ Pattern learning engine
├ storage/	✗ SQLite + vector search
└ sync/	✗ Anonymized sync protocol

Product 1: Furqan RaaS (Reasoning-as-a-Skill)

"Verified Reasoning for Any AI Agent"

Sprint R1 — MCP Server + Core Tools (Week 1-2)

R1.1 — MCP Server Skeleton

File: furqan-raas/mcp_server.py **Dependency:** pip install mcp

```

from mcp.server import Server
from mcp.types import Tool, TextContent

app = Server("furqan-reasoning")

@app.list_tools()
async def list_tools() -> list[Tool]:
    return [
        Tool(name="furqan_evaluate", ...),
        Tool(name="furqan_verify", ...),
        Tool(name="furqan_retrieve", ...),
        Tool(name="furqan_explain", ...),
        Tool(name="furqan_domains", ...),
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    ...

```

Steps:

1. Create furqan-raas/ directory as a standalone Python package
2. Install MCP SDK: `pip install mcp`
3. Implement MCP server with stdio transport
4. Wire tools to existing Orchestrator
5. Test with Claude Code / OpenClaw

Test: `python -m furqan_raas.mcp_server` starts without error. Claude Code can call `furqan_evaluate` .

R1.2 — Tool: furqan_evaluate

The main tool — full axiom-anchored evaluation.

```

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "furqan_evaluate":
        question = arguments["question"]
        domain = arguments.get("domain", "islamic")
        depth = arguments.get("depth", "standard")

        # 1. Get adapter for domain
        adapter = adapters.get(domain)

        # 2. Intent detection
        intent = detect_intent(question) # informational/evaluative/harmful

        # 3. If evaluative: full pipeline
        if intent == "evaluative":
            # LLM extracts facts
            extractions = chain_executor.extract(question, adapter)
            # Gates score deterministically

```

```

gate_scores = scorer.score_all(extractions)
# Z3 verifies
z3_result = verifier.verify_per_gate(extractions)
# LLM generates response
response = generate_response(question, gate_scores, z3_result)

return [TextContent(text=json.dumps({
    "type": "evaluation",
    "response": response,
    "verdict": {
        "gate_scores": gate_scores,
        "z3": z3_result,
        "total_score": avg_score,
    },
    "sources": sources,
})))]

elif intent == "informational":
    response = llm(question)
    return [TextContent(text=json.dumps({
        "type": "informational",
        "response": response,
    })))]

elif intent == "harmful":
    return [TextContent(text=json.dumps({
        "type": "refused",
        "response": "This request was refused based on safety analysis.",
        "reason": safety_reason,
    })))]

```

Steps:

1. Wire to existing `Orchestrator.evaluate()`
2. Add intent detection (informational/evaluative/harmful)
3. Add per-gate Z3 verification
4. Format response as structured JSON
5. Test with 10 benchmark questions

R1.3 — Tool: `furqan_verify`

Quick claim verification — lighter than full evaluate.

```

# Input: {"claim": "Honey is mentioned as healing in the Quran"}
# Output: {"verified": true, "confidence": 0.97, "sources": [...]}

```

Steps:

1. Search KB for matching sources
2. Grade confidence based on source strength
3. Return verification result + citations

4. No full gate evaluation — just source matching

R1.4 — Tool: `furqan_retrieve`

Pure knowledge retrieval — no evaluation.

```
# Input: {"query": "verses about patience", "domain": "islamic", "limit": 10}
# Output: {"sources": [...], "total_found": 23}
```

Steps:

1. Wire to existing `UnifiedRetriever.retrieve()`
 2. Format results with citations
 3. Support domain filtering
-

R1.5 — Tool: `furqan_explain`

Sourced explanation of a topic.

```
# Input: {"topic": "الربا", "depth": "detailed"}
# Output: {"explanation": "...", "sources": [...]}
```

Steps:

1. Retrieve relevant sources from KB
 2. Expand via Knowledge Graph
 3. Generate explanation via LLM grounded in sources
 4. Include full citations
-

R1.6 — Tool: `furqan_domains`

List available knowledge domains.

```
# Output: {"domains": [{"name": "islamic", "sources": 6316, "status": "active"},
...]}
```

R1.7 — SKILL.md Definition

File: `furqan-raas/SKILL.md`

Standard agent skill definition for OpenClaw/Claude Code discovery:

```
name: furqan-reasoning
description: >
  Axiom-anchored reasoning engine. Evaluates claims against
  verified sources with formal Z3 verification. Domain-agnostic.
commands:
  evaluate: Full 4-gate evaluation with Z3 proof
  verify: Quick claim verification against sources
  retrieve: Search verified knowledge bases
```

`explain`: Sourced explanation of a topic
`domains`: List available knowledge domains

R1.8 — Tests

```
tests/
├─ test_mcp_server.py          # MCP protocol tests
├─ test_tool_evaluate.py      # Full evaluation via MCP
├─ test_tool_verify.py        # Verification tool
├─ test_tool_retrieve.py      # Retrieval tool
├─ test_tool_explain.py       # Explanation tool
├─ test_intent_detection.py    # informational/evaluative/harmful routing
└─ test_safety_filter.py      # Harmful request detection
```

Sprint R2 — Knowledge Adapter System (Week 3-4)

R2.1 — Adapter Base Class

File: `furqan-raas/adapters/base.py`

```
from abc import ABC, abstractmethod

class KnowledgeAdapter(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...

    @property
    @abstractmethod
    def domain(self) -> str: ...

    @abstractmethod
    def retrieve(self, query: str, limit: int = 10) -> RetrievalResult: ...

    @abstractmethod
    def verify(self, claim: str) -> VerificationResult: ...

    @abstractmethod
    def get_axioms(self) -> DomainAxioms: ...
    # Domain axioms EXTEND core axioms, never replace
    # Validated against core via Z3 before acceptance
```

R2.2 — Islamic Adapter (First Implementation)

File: `furqan-raas/adapters/islamic.py`

Wraps existing KB layer:

```
class IslamicKnowledgeAdapter(KnowledgeAdapter):
    name = "al-furqan-islamic"
```

```

domain = "islamic"

def __init__(self, quran_collection, hadith_collection,
              fiqh_collection, graph_store, knowledge_linker):
    # Wraps existing Sprint 3 components
    ...

```

Steps:

1. Create adapter wrapper around existing collections
2. Implement retrieve() → wraps UnifiedRetriever
3. Implement verify() → wraps KB search + confidence scoring
4. Implement get_axioms() → returns Islamic domain axioms
5. Test adapter independently

R2.3 — Domain Router

File: furqan-raas/adapters/router.py

```

class DomainRouter:
    def __init__(self):
        self.adapters: dict[str, KnowledgeAdapter] = {}

    def register(self, adapter: KnowledgeAdapter) -> None:
        # Validate domain axioms against core via Z3
        self._validate_axioms(adapter)
        self.adapters[adapter.domain] = adapter

    def get(self, domain: str) -> KnowledgeAdapter:
        ...

    def _validate_axioms(self, adapter: KnowledgeAdapter) -> None:
        """Z3 check: domain axioms must not contradict core axioms."""
        # Uses SymbolicVerifier adapter contradiction checker
        ...

```

R2.4 — Adapter Template + Docs

File: furqan-raas/adapters/TEMPLATE.md

Guide for creating new domain adapters:

- Step-by-step instructions
- Data format requirements
- Axiom writing guide
- Testing checklist
- Example: Legal adapter skeleton

R2.5 — Tests

```
tests/
├─ test_islamic_adapter.py      # Islamic adapter wrapper
├─ test_domain_router.py       # Registration + routing
├─ test_axiom_validation.py     # Z3 contradiction check for domain axioms
└─ test_adapter_template.py    # Template adapter works
```

Sprint R3 — REST API + Remote Access (Week 5-6)

R3.1 — API Gateway

File: furqan-raas/api_gateway.py

FastAPI wrapper for remote MCP access:

```
from fastapi import FastAPI, Depends

app = FastAPI(title="Furqan RaaS API")

@app.post("/v1/evaluate")
async def evaluate(request: EvaluateRequest, api_key: str = Depends(verify_key)):
    ...

@app.post("/v1/verify")
async def verify(request: VerifyRequest, api_key: str = Depends(verify_key)):
    ...

@app.post("/v1/retrieve")
async def retrieve(request: RetrieveRequest, api_key: str = Depends(verify_key)):
    ...
```

R3.2 — SSE Transport for MCP

Remote MCP access via Server-Sent Events:

```
# Client config
mcpServers:
  furqan:
    url: "https://furqan.variance.com/mcp"
    transport: "sse"
    headers:
      Authorization: "Bearer ${FURQAN_API_KEY}"
```

R3.3 — API Key Management

- Per-domain API keys
- Usage tracking + billing hooks
- Rate limiting per key
- Reuse existing auth system from Sprint 2

R3.4 — SDK: Python

Package: furqan-sdk

```
from furqan import FurqanClient

client = FurqanClient(api_key="...")
result = client.evaluate("Is fractional reserve banking ethical?")
print(result.verdict)      # gate scores
print(result.response)     # natural language
print(result.z3_proof)     # formal proof
print(result.sources)      # citations
```

R3.5 — SDK: TypeScript

Package: @furqan/sdk

```
import { FurqanClient } from '@furqan/sdk';

const client = new FurqanClient({ apiKey: '...' });
const result = await client.evaluate('Is fractional reserve banking ethical?');
```

R3.6 — Tests

```
tests/
├─ test_api_gateway.py      # REST API endpoints
├─ test_sse_transport.py    # Remote MCP
├─ test_api_keys.py         # Auth + rate limiting
├─ test_python_sdk.py       # Python SDK
└─ test_typescript_sdk.py   # TS SDK (integration)
```

Sprint R4 — Production Hardening (Week 7-8)

R4.1 — Docker Packaging

```
FROM python:3.12-slim
COPY furqan-raas/ /app/
CMD ["python", "-m", "furqan_raas.api_gateway"]
```

R4.2 — Kubernetes Deployment

```
# Helm chart for Variance infra
apiVersion: apps/v1
kind: Deployment
metadata:
  name: furqan-raas
spec:
  replicas: 3
```


R4.3 — Monitoring + Observability

- Langfuse integration (LLM call tracing)
- Prometheus metrics (latency, token usage, gate distribution)
- Grafana dashboard
- Alert on anomalies (score distribution shift)

R4.4 — Documentation Site

- API reference (OpenAPI/Swagger)
- Getting started guide
- Adapter creation guide
- Pricing page

Product 2: Furqan Memory (Pattern Store Skill)

"Your AI Agent Learns and Remembers"

Sprint M1 — Core Memory Engine (Week 1-2)

M1.1 — SQLite Storage Layer

File: furqan-memory/storage/sqlite_store.py

```
import sqlite3

class MemoryStore:
    def __init__(self, db_path: str = "furqan_memory.db"):
        self.db = sqlite3.connect(db_path)
        self._create_tables()

    def _create_tables(self):
        """Create verdicts, patterns, feedback, context tables."""
        self.db.executescript("""
            CREATE TABLE IF NOT EXISTS verdicts (
                id TEXT PRIMARY KEY,
                question TEXT NOT NULL,
                domain TEXT DEFAULT 'islamic',
                verdict_json TEXT NOT NULL,
                total_score INTEGER,
                gate_results TEXT,
                created_at REAL,
                accessed_at REAL,
                access_count INTEGER DEFAULT 0
            );

            CREATE TABLE IF NOT EXISTS patterns (
                id TEXT PRIMARY KEY,
                category TEXT NOT NULL,
                domain TEXT DEFAULT 'islamic',
                rule TEXT NOT NULL,
                signals TEXT NOT NULL,
                expected_gates TEXT NOT NULL,
```

```

        template TEXT,
        confidence REAL DEFAULT 0.3,
        source_verdicts TEXT,
        hit_count INTEGER DEFAULT 0,
        last_hit REAL,
        created_at REAL,
        version INTEGER DEFAULT 1
    );

    CREATE TABLE IF NOT EXISTS feedback (
        id TEXT PRIMARY KEY,
        target_id TEXT NOT NULL,
        target_type TEXT NOT NULL,
        rating TEXT NOT NULL,
        correction TEXT,
        created_at REAL
    );
    """

```

M1.2 — Embedded Vector Search

File: furqan-memory/storage/vector_store.py

```

# Option A: sqlite-vss (zero external dependency)
# Option B: chromadb embedded (already in project)

class VectorSearch:
    def __init__(self, embedding_model):
        self.embedder = embedding_model
        # Use ChromaDB embedded for MVP
        self.collection = chromadb.Client().create_collection("memory")

    def add(self, id: str, text: str, metadata: dict): ...
    def search(self, query: str, limit: int = 5) -> list: ...

```

M1.3 — Verdict Memory (remember + recall)

File: furqan-memory/memory_manager.py

```

class MemoryManager:
    def __init__(self, store: MemoryStore, vectors: VectorSearch):
        self.store = store
        self.vectors = vectors

    def remember(self, verdict: dict, tags: list = None) -> str:
        """Store a verdict in local memory."""
        verdict_id = generate_id()
        self.store.save_verdict(verdict_id, verdict)
        self.vectors.add(verdict_id, verdict["question"], verdict)
        # Trigger async pattern extraction
        self._maybe_extract_pattern(verdict)

```

```
        return verdict_id

def recall(self, query: str, limit: int = 5) -> list:
    """Search memory for relevant past verdicts."""
    return self.vectors.search(query, limit)
```

M1.4 — MCP Server (Client-Side)

File: furqan-memory/mcp_server.py

```
app = Server("furqan-memory")

@app.list_tools()
async def list_tools():
    return [
        Tool(name="furqan_remember", ...),
        Tool(name="furqan_recall", ...),
        Tool(name="furqan_recognize", ...),
        Tool(name="furqan_reflect", ...),
        Tool(name="furqan_feedback", ...),
        Tool(name="furqan_memory_stats", ...),
    ]
```

M1.5 — SKILL.md

```
name: furqan-memory
description: >
    Client-side memory and learning skill. Stores verdicts,
    learns patterns, enables offline reasoning.
    All data stays on user's device.
commands:
    remember: Store a verdict in local memory
    recall: Search memory for relevant past verdicts
    recognize: Fast-path pattern matching (<50ms)
    reflect: Learn patterns from recent verdicts
    feedback: Rate a verdict to improve accuracy
    stats: Memory usage statistics
```

M1.6 — Tests

```
tests/
├─ test_sqlite_store.py      # Storage CRUD
├─ test_vector_search.py    # Semantic search
├─ test_memory_manager.py   # remember + recall
├─ test_mcp_memory.py       # MCP protocol
└─ test_skill_definition.py # SKILL.md validation
```

Sprint M2 — Pattern Learning (Week 3-4)

M2.1 — Pattern Extractor

File: furqan-memory/patterns/extractor.py

```
class PatternExtractor:
    def extract(self, verdict: dict) -> Pattern:
        """Extract a generalized pattern from a verdict."""
        signals = self._extract_signals(verdict)
        expected_gates = self._extract_gate_expectations(verdict)
        template = self._build_template(verdict)

        return Pattern(
            category=verdict["primary_system"],
            signals=signals,
            expected_gates=expected_gates,
            template=template,
            confidence=0.3, # New pattern starts low
        )

    def _extract_signals(self, verdict: dict) -> list[str]:
        """Extract keywords/concepts that trigger this pattern."""
        # Use embedding similarity to find defining terms
        ...
```

M2.2 — Pattern Matcher (recognize)

File: furqan-memory/patterns/matcher.py

```
class PatternMatcher:
    def recognize(self, query: str, threshold: float = 0.75) -> PatternMatch:
        """Check if query matches a known pattern. Target: <50ms."""
        # 1. Embed query
        # 2. Search pattern vectors
        # 3. If similarity > threshold and confidence > 0.8:
        #     -> Return pattern match (fast path!)
        # 4. Else: no match (proceed to full evaluation)
        ...
```

M2.3 — Pattern Lifecycle Manager

File: furqan-memory/patterns/lifecycle.py

```
class PatternLifecycle:
    """Manages pattern confidence over time."""

    def evolve(self, pattern: Pattern, new_verdict: dict) -> Pattern:
        """Update pattern confidence based on new verdict."""
        if self._consistent(pattern, new_verdict):
            pattern.confidence = min(1.0, pattern.confidence + 0.15)
            pattern.hit_count += 1
```

```

    else:
        pattern.confidence = max(0.0, pattern.confidence - 0.1)
    return pattern

def decay(self, patterns: list[Pattern], max_age_days: int = 90):
    """Archive patterns with no hits in max_age_days."""
    ...

```

M2.4 — Reflect Command

File: Part of `memory_manager.py`

```

def reflect(self, since: str = None) -> ReflectionResult:
    """Batch pattern extraction from recent verdicts."""
    recent = self.store.get_verdicts_since(since or last_24h)
    new_patterns = 0
    updated_patterns = 0

    for verdict in recent:
        existing = self.matcher.find_similar_pattern(verdict)
        if existing:
            self.lifecycle.evolve(existing, verdict)
            updated_patterns += 1
        else:
            pattern = self.extractor.extract(verdict)
            self.store.save_pattern(pattern)
            new_patterns += 1

    return ReflectionResult(new=new_patterns, updated=updated_patterns)

```

M2.5 — Tests

```

tests/
├─ test_pattern_extractor.py      # Signal extraction
├─ test_pattern_matcher.py       # Recognition + threshold
├─ test_pattern_lifecycle.py     # Confidence evolution + decay
├─ test_reflect.py               # Batch learning
└─ test_determinism.py           # Same verdict → same pattern (10x test)

```

Sprint M3 — Feedback + Privacy (Week 5-6)

M3.1 — Feedback Collection

```

def feedback(self, id: str, rating: str, correction: str = None):
    """Rate a verdict/pattern. Adjusts confidence."""
    self.store.save_feedback(id, rating, correction)

    if rating == "incorrect":
        pattern = self.store.get_pattern_for_verdict(id)

```

```
if pattern:
    pattern.confidence -= 0.3
    self.store.update_pattern(pattern)
```

M3.2 — Export/Import

```
def export(self, format: str = "json") -> str:
    """Export all memory data for backup/migration."""
    ...

def import_data(self, path: str) -> int:
    """Import memory data from backup."""
    ...
```

M3.3 — Anonymized Sync Protocol (Opt-in)

```
class SyncManager:
    def push(self, domain: str, anonymize: bool = True):
        """Push mature patterns to server (opt-in)."""
        patterns = self.store.get_mature_patterns(min_confidence=0.8)
        if anonymize:
            patterns = self._strip_pii(patterns)
        self.api.push_patterns(patterns)

    def pull(self, domain: str):
        """Pull community patterns from server."""
        community = self.api.get_community_patterns(domain)
        self.store.import_patterns(community)
```

M3.4 — SQLCipher Encryption

```
# Optional encryption at rest
import pysqlcipher3

class EncryptedMemoryStore(MemoryStore):
    def __init__(self, db_path: str, key: str):
        self.db = pysqlcipher3.connect(db_path)
        self.db.execute(f"PRAGMA key = '{key}'")
```

M3.5 — Tests

```
tests/
├─ test_feedback.py           # Rating + confidence adjustment
├─ test_export_import.py     # Data portability
├─ test_sync.py              # Push/pull protocol
├─ test_anonymization.py     # PII stripping
```

```
|─ test_encryption.py          # SQLCipher
|─ test_privacy.py            # Verify no PII leakage
```

Sprint M4 — Production (Week 7-8)

M4.1 — PyPI Package

```
pip install furqan-memory
```

M4.2 — npm Package

```
npm install @furqan/memory
```

M4.3 — Performance Benchmarks

Operation	Target
recognize	< 50ms
recall	< 200ms
remember	< 100ms
reflect	< 2s

M4.4 — Documentation

- Getting started guide
- Integration with RaaS
- Privacy policy template
- Storage limits guide

Agent Assignment

Agent	Skill	Sprints	Key Deliverables
Agent-A	RaaS MCP	R1	MCP server + 5 tools
Agent-B	RaaS Adapters	R2	Adapter system + Islamic adapter
Agent-C	RaaS API	R3	REST API + SDKs
Agent-D	Memory Core	M1	SQLite + vectors + MCP
Agent-E	Memory Patterns	M2	Extractor + matcher + lifecycle
Agent-F	Memory Privacy	M3	Feedback + sync + encryption
Agent-G	Production	R4 + M4	Docker + K8s + docs

Parallel Execution Strategy

Week 1-2:	Agent-A: R1 (RaaS MCP)		Agent-D: M1 (Memory Core)
Week 3-4:	Agent-B: R2 (Adapters)		Agent-E: M2 (Patterns)
Week 5-6:	Agent-C: R3 (API + SDKs)		Agent-F: M3 (Privacy)
Week 7-8:	Agent-G: R4 + M4 (Production) – both skills		

Total: 8 weeks, 7 agents, 2 commercial products.

Commercial Model

Component	Pricing	Rationale
Furqan Memory	Free / Open Source	Runs on user's device → drives adoption
Furqan RaaS	Per-evaluation	Server resources + LLM costs + KB curation
Community Patterns	Free tier + Premium	Basic patterns free, specialized paid
Custom Adapters	Enterprise license	Domain-specific knowledge bases
Self-hosted RaaS	Enterprise license	On-premise deployment

Strategy: Memory = trojan horse (free, local, useful) → drives adoption of RaaS (paid).

Success Metrics

Metric	Target	Timeframe
MCP tools working	5/5	Week 2
Islamic adapter live	✅	Week 4
Second domain adapter	Legal or Medical	Week 8
API response time	< 15s (evaluation)	Week 6
Pattern recognition	< 50ms	Week 4
Memory storage	< 500MB for 10K verdicts	Week 2
SDK packages published	Python + TypeScript	Week 6
Docker image	< 500MB	Week 8
Agent adoption	3+ agents using	Month 3

This plan builds on the existing AI-Furqan engine (Sprints 3-5) and packages it as two commercial products. Ready for multi-agent execution.